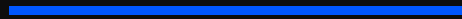


LIVE TASK

SecureGate

Build a Focused Authentication & Security App



Design to MVP Bootcamp · Assessment Task · Individual Submission

Stack: Next.js · TypeScript · Prisma · PostgreSQL · NextAuth · Resend · Vercel

This task tests whether you can build something real, think like a product engineer, and explain the software laws that govern every decision you make.

DURATION	TIME LIMIT	SUBMISSION	SOLO/TEAM
3 Hours	Strictly Enforced	GitHub + Vercel + Markdown	Solo

01. Overview

Over the past weeks, you have learned how to build real products. You have set up authentication, connected databases, sent transactional emails, integrated payments, and used AI agents to scaffold full-stack applications with Next.js, TypeScript, Prisma, and PostgreSQL. You have also learned the software engineering laws that every serious builder must internalise.

This live task is not about building a complete product. It is about going deep on one thing: **authentication and security**. You will build **SecureGate** — a focused, production-ready authentication system as a standalone Next.js app. Small scope. Deep execution. Zero shortcuts.

What is SecureGate?

SecureGate is a standalone authentication app. It is not a full product. It has one job: to show that you understand how to build identity and access management the right way, using the tools and engineering principles you have been taught. Think of it as the auth layer you would extract from any serious application and study under a microscope.

What you will build

- **Sign Up page** — with full form validation, password strength indicator, and email confirmation via Resend
- **Login page** — with email + password, NextAuth session handling, and proper error messaging that does not leak security information
- **Email Verification flow** — user receives a token link, clicks it, account is verified in the database
- **Protected Dashboard page** — only accessible to verified, authenticated users. Redirect logic must be airtight
- **Forgot Password flow** — request a reset, receive an email, submit a new password. Token must expire
- **Rate Limiting middleware** — brute-force protection on the login endpoint
- **Logout** — clean session destruction, redirect to login
- **Password hashing** — using bcrypt with a proper salt round configuration

Murphy's Law is watching you.

Anything that can go wrong in an auth system will go wrong. Your job is to build as if Murphy himself is a user of SecureGate. Rate limit your endpoints. Hash your passwords. Expire your tokens. Sanitise your inputs. Do not trust what comes from the client. Every gap you leave is a gap Murphy will find.

02. Tech Stack & Setup

Layer	Tool	Purpose
Framework	Next.js 14 (App Router)	Full-stack React framework
Language	TypeScript	Type safety across the codebase
Database	PostgreSQL via Prisma ORM	User table, tokens, sessions
Auth	NextAuth.js (Auth.js)	Session management, providers
Password	bcrypt (via bcryptjs)	Secure password hashing
Email	Resend + React Email	Verification + reset emails
Validation	Zod	Server-side input validation schemas
Rate Limit	upstash/ratelimit or custom middleware	Brute-force protection
Deployment	Vercel	Production hosting + env vars
Repo	GitHub	Version control, required for submission

Environment Variables

Create a `.env.local` file at the root of your project. Never commit this file. Add it to `.gitignore` before your first push. Your Vercel deployment must use the dashboard environment variables panel — not hardcoded values.

```
DATABASE_URL
NEXTAUTH_SECRET
NEXTAUTH_URL
RESEND_API_KEY
UPSTASH_REDIS_REST_URL (if using Upstash rate limiting)
UPSTASH_REDIS_REST_TOKEN
```

Kerckhoffs's Principle — your system's security must not depend on secrecy of its design.

This means: do not hide your code and call that security. Your security must come from the strength of your hashing, the integrity of your tokens, and the correct use of secrets stored in environment variables — not from hiding your implementation.

03. Build Phases

Follow these phases in order. Each phase has a clear goal. Do not skip ahead. A broken phase 2 built on a shaky phase 1 is worse than a solid phase 1 alone.

PHASE 1

Scaffold & Database Schema

- Bootstrap a new Next.js 14 project with the App Router and TypeScript
- Initialise Prisma and connect to your PostgreSQL database
- Create a User model with fields: id, name, email, password (hashed), emailVerified, createdAt
- Create a VerificationToken model: identifier, token, expires
- Create a PasswordResetToken model: email, token, expires
- Run prisma migrate dev to apply the schema. Confirm tables in your DB client
- Push your initial scaffold to GitHub before writing any feature code

TOOLS: Prisma, PostgreSQL, GitHub Desktop, Windsurf / Cursor

PHASE 2

Authentication Core with NextAuth

- Install and configure NextAuth with the Credentials provider
- Implement the authorize() function — query the user by email, compare hashed password using bcryptjs.compare()
- Set up the session strategy (JWT or database sessions — your choice, but justify it in your markdown)
- Create the Sign Up API route: validate with Zod, hash password using bcrypt.hash() with salt rounds of 12, save user
- Protect the /dashboard route using NextAuth middleware — redirect unauthenticated users to /login
- Test sign up and sign in manually. Confirm the session is created. Confirm the password in the DB is NOT plain text

TOOLS: NextAuth.js, bcryptjs, Zod, Prisma

PHASE 3

Email Verification Flow

- On sign up, generate a secure random token using `crypto.randomBytes(32).toString('hex')`
- Save the token and an expiry (15 minutes) to the `VerificationToken` table
- Send a verification email using Resend with the token URL embedded
- Create the `/verify-email/[token]` route: look up the token, check expiry, mark user as verified, delete token
- If token is expired or not found, show a clear error and offer a 'resend verification' option
- Only verified users should be able to access the dashboard — add this check to your session/middleware logic

TOOLS: Resend, React Email, crypto (Node built-in), Prisma

PHASE 4

Forgot Password Flow

- Build a `/forgot-password` page with an email input field
- On submit: look up the email, generate a reset token, save it with a 1-hour expiry to `PasswordResetToken`
- Send a password reset email via Resend with the token link
- Build the `/reset-password/[token]` route: validate token, check expiry, accept new password
- Hash the new password before saving. Delete the used token. Redirect user to login
- Important: if the email is not found, still return a success message — do not confirm whether the email exists

TOOLS: Resend, bcryptjs, Prisma, crypto

PHASE 5

Rate Limiting & Security Hardening

- Add rate limiting to the `POST /api/auth/signin` endpoint — maximum 5 attempts per IP per 10 minutes
- Add rate limiting to the `/forgot-password` submission endpoint
- Review all your API error messages: they must not reveal whether an email exists, what the password was, or internal stack traces
- Ensure all sensitive environment variables are correctly configured in Vercel — not hardcoded
- Add HTTP security headers in `next.config.js`: `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`
- Run your app and try to break it yourself: wrong password, expired token, missing fields. Document what happens

TOOLS: Upstash Redis (or custom middleware), `next.config.js`, Vercel

PHASE 6

UI Polish & Deployment

- Your UI must be clean, usable, and consistent — apply the design principles you have learned
- Every form must have accessible labels, real validation messages (not just 'Something went wrong'), and a loading state
- The password field must show a strength indicator (weak / fair / strong based on length and character variety)
- Deploy to Vercel. Set all environment variables in the Vercel dashboard. Test the live URL end to end
- Push your final code to GitHub. Confirm your .env.local is NOT in the repo
- Record your live Vercel URL — you will submit it

TOOLS: Vercel, GitHub, Figma (reference), Tailwind CSS

04. Engineering Laws Quiz

This is where you prove you are not just a builder — you are a thinking engineer. For each question below, you must write a clear, honest answer in your **REFLECTION.md** file. Reference specific parts of your SecureGate code. Vague or generic answers will not earn marks.

Submission Rule

Your REFLECTION.md file must contain all 15 answers, numbered to match the questions here. Each answer must include: (1) your explanation in plain English, (2) a code snippet or file path from your SecureGate repo, and (3) what could go wrong if this was ignored.

01

Where in SecureGate did Murphy's Law force you to add protection you would not have thought about otherwise? Name at least two specific places and explain what could have gone wrong.

Hint: Think about password brute force, token expiry, missing email checks, session edge cases.

LAW IN FOCUS: Murphy's Law — Anything that can go wrong will go wrong

02

NextAuth, Prisma, and Resend are all abstractions. Pick one and explain where it 'leaks' — where you had to understand the layer beneath it to make something work correctly.

Hint: Think about when the abstraction surprised you, forced you to read raw docs, or failed silently.

LAW IN FOCUS: The Law of Leaky Abstractions — All non-trivial abstractions, to some degree, are leaky

03

SecureGate intentionally does not have social login, multi-factor auth, or audit logs. Explain why adding those features right now would violate YAGNI, and how you would add them correctly later.

Hint: Think about what is actually required for this task versus what sounds impressive.

LAW IN FOCUS: YAGNI — You Aren't Gonna Need It

04

Look at your password hashing implementation. What is a salt, why does bcrypt use it automatically, and what would happen to your users if you stored SHA-256 hashes instead?

Hint: Think about rainbow tables, dictionary attacks, and why bcrypt is slow by design.

LAW IN FOCUS: Kerckhoffs's Principle — Security must not rely on the secrecy of the algorithm

05

Your forgot-password endpoint returns a success message even if the email does not exist. Why? What law or principle governs this decision, and what would happen to user privacy if you changed it?

Hint: Think about what an attacker learns if the response differs based on whether the email exists.

LAW IN FOCUS: Postel's Law + Security by Design — Be conservative in what you send

06

Find one place in your codebase where you applied the Boy Scout Rule — where you cleaned up something that was not part of your original plan. What did you find? What did you fix?

Hint: It could be a variable name, a duplicated function, an unused import, or a messy component.

LAW IN FOCUS: The Boy Scout Rule — Leave the code better than you found it

07

Your SecureGate started as a scaffold and grew phase by phase. How does this match Gall's Law? What would have happened if you tried to build all six phases at the same time?

Hint: Think about dependency order, debugging surface area, and how bugs compound.

LAW IN FOCUS: Gall's Law — A complex system that works evolved from a simple system that worked

08

You built SecureGate using Prisma to talk to PostgreSQL. Identify one situation where the Prisma schema model and the actual database table structure are NOT the same thing. Why does this matter?

Hint: Think about generated fields, relation tables, cascades, or how Prisma handles optional vs required.

LAW IN FOCUS: The Law of Leaky Abstractions — applied to ORMs specifically

09

Rate limiting is not in the core Next.js or NextAuth package. You had to add it yourself. What software engineering principle does this demonstrate, and how would Zawinski's Law warn you about what happens when apps grow without discipline?

Hint: Think about feature creep, responsibility boundaries, and single-purpose design.

LAW IN FOCUS: Zawinski's Law — Every program attempts to expand until it can read mail

10

Your login form shows an error message when credentials are wrong. What exact message do you show, and why did you choose that specific wording? What would the Principle of Least Surprise say about how error messages should behave?

Hint: Think about what users expect to see when they enter a wrong password versus a wrong email.

LAW IN FOCUS: The Principle of Least Surprise — Software should behave in the way users expect

11

Look at your /dashboard route protection. How does your middleware know the user is authenticated? If a user manually deletes their session cookie, what happens? Trace the exact code path.

Hint: Think about where the check happens, what it reads, and what the fallback behaviour is.

LAW IN FOCUS: Murphy's Law + Defensive Programming — Assume the worst-case user

12

You used environment variables to store secrets. Explain what would happen — step by step — if your NEXTAUTH_SECRET was accidentally committed to GitHub and how you would recover from it.

Hint: Think about secret rotation, invalidating old sessions, and what attackers can do with a leaked NextAuth secret.

LAW IN FOCUS: Kerckhoffs's Principle + Technical Debt — Security debt has compounding interest

13

SecureGate required you to write code across routes, middleware, database schema, and email templates. How does Conway's Law explain why full-stack developers organise code the way they do? How is your folder structure a reflection of how you think?

Hint: Think about how the structure of a team or a mind shapes the structure of a system.

LAW IN FOCUS: Conway's Law — Systems mirror the communication structure of the people who build them

14

Identify one piece of technical debt in your SecureGate codebase — something that works right now but will cause problems when the app grows. Describe the debt precisely, explain why you left it, and write the refactored version.

Hint: It could be a hardcoded value, a missing abstraction, a route handler that does too much, or a type that should be shared.

LAW IN FOCUS: Technical Debt — Everything that slows you down later because of a shortcut taken now

15

If you were asked to add Flutterwave payment integration to SecureGate — so users pay to unlock a premium dashboard — walk through every engineering principle from this task that would still apply. Which ones become more critical when money is involved?

Hint: Think about what payments demand from authentication, security, idempotency, error handling, and user trust.

LAW IN FOCUS: Synthesis question — All principles apply

05. REFLECTION.md — Required Structure

You must include a file named `REFLECTION.md` in the root of your GitHub repository. This file is worth 40% of your total score. It must follow the structure below exactly.

```
# SecureGate — Reflection & Engineering Analysis
**Name:** [Your full name]
**Cohort:** Design to MVP Bootcamp
**Live URL:** [Your Vercel deployment link]
**GitHub Repo:** [Your repo URL]

---

## Part 1 — What I Built
[2-3 sentences describing SecureGate and the specific features you implemented]

## Part 2 — What Surprised Me
[The one thing that was harder than expected, and what you learned from it]

## Part 3 — Engineering Laws Quiz

### Q1 — Murphy's Law
**Code reference:** `src/app/api/auth/[...nextauth]/route.ts` lines 34-48
**My Answer:** [Your answer here]
**What goes wrong if ignored:** [Your answer here]

### Q2 — Law of Leaky Abstractions
... [repeat this pattern for all 15 questions]

## Part 4 — One Thing I Would Refactor
[Describe your identified technical debt and paste the refactored version]

## Part 5 — How This Changes How I Build
[What you now know about authentication, security, and engineering principles that you did not know before]
```

Writing Standard

Your `REFLECTION.md` must be written in plain English. No bullet-point lists of copied definitions. No pasting from Wikipedia or the course slides. Every answer must reference your actual SecureGate code. If your answer could apply to any project by any student, it is not specific enough.

06. Submission Checklist

Before you submit, go through this list item by item. Do not guess. Open the live URL. Check the repo. Read the markdown file.

☐ App is live on Vercel

Open the URL in an incognito window. Does sign up work from scratch?

☐ Sign up creates a verified user in the DB

Check your database — is emailVerified set after clicking the email link?

☐ Passwords are hashed in the database

Open your DB client. Is the password column a bcrypt hash starting with \$2b\$?

☐ Verification email sends correctly

Sign up with a real email. Does the email arrive? Does the link work?

☐ Forgot password flow works end to end

Request a reset. Receive the email. Set a new password. Log in with it.

☐ Rate limiting is active on login

Make 6 wrong login attempts in quick succession. Are you blocked on attempt 6?

☐ Protected route redirects correctly

Log out. Try accessing /dashboard directly. Do you land on /login?

☐ No .env.local in the GitHub repo

Open your GitHub repo. Search for .env.local. It must not appear.

☐ REFLECTION.md is in the repo root

Open GitHub. Is REFLECTION.md there? Are all 15 questions answered?

☐ No hardcoded API keys or secrets in code

Search your codebase for your actual API key strings. Find zero results.

☐ All environment variables set in Vercel

Open Vercel > Settings > Environment Variables. Are all vars present?

☐ UI has loading states and real error messages

Submit an empty form. What happens? Submit wrong credentials. What shows?

07. Scoring Rubric

Your submission will be assessed across six dimensions. Each area is scored 1–5. Maximum total: **30 points**.

Area	Excellent (5)	Acceptable (3)	Needs Work (1)
App Functionality	Auth works end-to-end, emails send, correct roles, no bugs present	Auth works, some minor bugs present	Auth broken or incomplete
Security Depth	Hashing, rate-limiting, env vars, no leaks	Hashing done, some gaps remain	Passwords stored plain or major gaps
Code Quality	Abstracted, no duplication, clean follow-through	Messy code, some repeated code	Spaghetti, no structure
Engineering Laws Documentation	All 15 questions answered with code references	8–14 answers with some code refs	Fewer than 8 answered
Deployment	Live on Vercel, .env correct, no keys exposed	Live on Vercel, some env issues or minor leaks	Not deployed
Markdown File	Well-structured, clear, honest, insightful	Present and readable	Missing or mostly blank

Score Range	Grade	What It Means
27–30	Distinction	You have internalised the principles. This code could go to production.
21–26	Merit	Strong execution. Minor gaps in depth or documentation.
15–20	Pass	You built it. Reflection is shallow. Security gaps present.
Below 15	Resubmit	Critical features missing or principles not understood.

What Distinction Looks Like

A distinction submission is one where you could sit in front of a senior engineer, open your REFLECTION.md, and walk them through your choices line by line. You know why you chose 12 salt rounds. You know what happens when the reset token expires. You know which law told you to not confirm whether an email exists. That is the standard.

08. Submission Instructions

1

GitHub Repository URL

Your complete codebase. Must include REFLECTION.md in the root. Must NOT include .env.local or any secrets.

2

Vercel Live URL

A working deployment. We will test it cold — no prior sign-in. Sign up, verify email, log in, access dashboard, reset password.

3

Quick Voice Note (2 min max)

Record a 2-minute voice note where you answer one quiz question out loud. Any question. Any format. This confirms the reflection is yours.

Integrity Notice

You may use AI agents (Windsurf, Cursor, Claude, Gemini) to help you build SecureGate. This is expected. What is not acceptable is submitting a REFLECTION.md written by an AI. Your engineering analysis must be yours. We will ask follow-up questions on your submission. If you cannot explain what you submitted, the grade does not stand.

Build it properly. Document it honestly. Ship it with pride. That is what a product engineer does.